Inverse Problems with Diffusion Models: A MAP Estimation Perspective

Sai Bharath Chandra Gutha[1], Ricardo Vinuesa[2], Hossein Azizpour[1]
[1]RPL, KTH Royal Institute of Technology, Sweden
[2]FLOW, KTH Royal Institute of Technology, Sweden
sbcgutha@kth.se, rvinuesa@mech.kth.se, azizpour@kth.se

Abstract

Inverse problems have many applications in science and engineering. In Computer vision, several image restoration tasks such as inpainting, deblurring, and super-resolution can be formally modeled as inverse problems. Recently, methods have been developed for solving inverse problems that only leverage a pre-trained unconditional diffusion model and do not require additional task-specific training. In such methods, however, the inherent intractability of determining the conditional score function during the reverse diffusion process poses a real challenge, leaving the methods to settle with an approximation instead, which affects their performance in practice. Here, we propose a MAP estimation framework to model the reverse conditional generation process of a continuous time diffusion model as an optimization process of the underlying MAP objective, whose gradient term is tractable. In theory, the proposed framework can be applied to solve general inverse problems using gradient-based optimization methods. However, given the highly non-convex nature of the loss objective, finding a perfect gradient-based optimization algorithm can be quite challenging, nevertheless, our framework offers several potential research directions. We use our proposed formulation to develop empirically effective algorithms for image restoration. We validate our proposed algorithms with extensive experiments over multiple datasets across several restoration tasks.

1. Introduction

Inverse problems are ubiquitous in science and engineering with a wide range of downstream applications [2, 28]. In Computer vision, several image restoration tasks such as inpainting, deblurring, super-resolution, and more, can be formally modeled as inverse problems. In an inverse problem, characterized by Eq. (1), $y \in \mathbb{R}^m$ is a (potentially noisy) observation of the original data $x \in \mathbb{R}^n$, and η is a random variable denoting i.i.d. noise, typically assumed to be Gaussian with a known variance i.e $\eta \sim \mathcal{N}(0, \sigma_y^2 \mathbb{I})$, and the task is to infer the original data x given the observation y. The function $\mathcal{A} : \mathbb{R}^n \to \mathbb{R}^m$ is known as the forward operator, and typically $n \gg m$, indicating that the observation $y \in \mathbb{R}^m$ corresponds to a severely degraded signal, from which one needs to recover the original signal $x \in \mathbb{R}^n$, which makes the task highly challenging. For linear inverse problems, $\mathcal{A}$ denotes a linear mapping and can be substituted with a matrix $H \in \mathbb{R}^{m \times n}$.

$$y = \mathcal{A}(x) + \eta \tag{1}$$

Several conventional approaches for solving inverse problems exist [1]. These include approaches based on functional-analytic, probabilistic, data-driven methods, and more. Recently, Deep Learning (DL) based methods have been applied to solve inverse problems and have shown great success. In a Bayesian framework, solving an inverse problem naturally corresponds to estimating the posterior $P(x|y)$. Typical DL-based approaches for solving inverse problems fall into two categories. **1.** Methods that directly learn the posterior $P(x|y)$ via conditional generative models [14, 19], and **2.** Methods that learn $P(x)$ via an unconditional generative model and use it to infer $P(x|y)$ [5, 12, 21, 29]. Methods of the former category require task-specific training, i.e. training with a dataset of pairs (x, y), where the degradation y is computed using x and a task-specific forward operator $\mathcal{A}$. This limits the out-of-the-box applicability of the model to a different task (different forward operator). On the contrary, methods of the latter category train an unconditional generative model to learn $P(x)$, and this training is task-independent since it only needs a dataset of original data samples x. These methods then use the trained model for $P(x)$ and since $P(y|x)$ is tractable (i.e. from Eq. (1), $P(y|x) = \mathcal{N}(\mathcal{A}(x), \sigma_y^2 \mathbb{I})$), utilizing the Bayes rule, they infer the posterior $P(x|y) \propto P(y|x)P(x)$.

Several choices for Deep Generative Models (DGMs) exist, each with its advantages and disadvantages. There have been approaches using Generative Adversarial Networks (GAN) [9] and Normalizing Flow (NF) [17] based

DGMs for solving inverse problems, with more recent methods focusing on Diffusion models [10, 20, 26], owing to their state-of-the-art performance in several vision-based generative tasks. This work focuses on methods that use a pre-trained unconditional diffusion model as the prior $P(x)$ and infer the posterior $P(x|y)$ for solving inverse problems. Sec. 2 provides some background on diffusion models and related works that use unconditional diffusion models for solving inverse problems and their inherent limitations. Later in Sec. 3, we propose our Maximum A Posteriori (MAP) estimation framework for continuous-time diffusion models and discuss the practical implementation. Specifically, we propose a novel MAP formulation that employs a reparameterization based on consistency models [23] to model the reverse conditional diffusion process as MAP optimization. In Sec. 4, we use our proposed framework to develop empirically effective algorithms for image restoration, and in Sec. 5, we validate the algorithms with extensive experiments on deblurring, super-resolution, and image inpainting. In Sec. 6, we present a brief discussion before concluding our findings in Sec. 7.

2. Background

Given a dataset $\mathcal{D} = \{x_i\}_{i=1}^N$, where each x_i is an i.i.d. sample drawn from an unknown data distribution $P_{\text{data}}(x)$, a DGM learns to approximate P_{data} from the samples in $\mathcal{D}$.

2.1. Diffusion Models

Diffusion models [10, 20, 26] are a recent family of generative models. The methodology involves simulating a stochastic process $\{x(t)\}_{t=0}^T$ described by a Stochastic Differential Equation (SDE) such as Eq. (2), where $t \in [0, T]$ is a continuous time variable, $x(0) \sim P_0 = P_{\text{data}}$ is the data distribution for which we have a dataset $\mathcal{D}$ of samples, $x(T) \sim P_T$ is a tractable prior distribution. The functions $f(\cdot, t) : \mathbb{R}^n \to \mathbb{R}^n$ and $g(\cdot) : \mathbb{R} \to \mathbb{R}$ are called the drift and diffusion coefficients of $x(t)$ respectively, and dw denotes the standard Wiener process. Typically f and g are chosen in a way that yields a tractable prior P_T which contains no information about P_0 (i.e. P_{data}).

$$dx = f(x, t)dt + g(t)dw \tag{2}$$

Eq. (2) also describes the "forward process", in which, starting from an initially clean data sample, a small amount of noise is progressively added at each step until it turns into a noisy sample of the prior distribution P_T. The backward/reverse process which transforms a noisy sample of P_T into a clean sample of the data distribution is described by the corresponding reverse-SDE in Eq. (3)

$$dx = [f(x, t) - g(t)^2 \nabla_x \log P_t(x)]dt + g(t)d\bar{w} \tag{3}$$

$d\bar{w}$ denotes the standard Wiener process when t flows backwards from T to 0 with dt denoting an infinitesimal negative time step. The term $\nabla_x \log P_t(x)$ is called the score function of the marginal distribution $P_t(x)$. If we know this score function for each marginal distribution i.e. for all t, then one could solve the reverse-SDE in Eq. (3) and generate samples from the data distribution. In general, the score function is not analytically tractable and is hard to estimate, however, one could train a time-indexed neural network model to learn the score function via score matching techniques [25, 27]. The trained score model $S_\theta(x, t)$ can be substituted in place of $\nabla_x \log P_t(x)$ in Eq. (3), and the reverse-SDE can be solved using traditional SDE solvers [26].

$$dx = [f(x, t) - \frac{1}{2} g(t)^2 \nabla_x \log P_t(x)]dt \tag{4}$$

For the stochastic process described by the SDE in Eq. (2), there exists a corresponding Ordinary Differential Equation (ODE) shown in Eq. (4), describing a deterministic process whose trajectories share the same marginal probability densities $\{P(x(t))\}_{t=0}^T$ as those simulated by the SDE. This is called the Probability Flow ODE (PF ODE) in the literature [26]. So equivalently, one could also use ODE solvers to solve Eq. (4) in reverse time from $t = T$ until 0 to generate samples from the data distribution.

Hereon, we assume the default choice for drift and diffusion coefficients as $f(x, t) = 0$ and $g(t) = \sqrt{\frac{\mathrm{d}\sigma^2(t)}{\mathrm{d}t}}$, where $\sigma(t)$ is a monotonically increasing noise schedule from $t = 0$ to T, with $\sigma(T)$ being very high. This choice of f and g results in a closed form perturbation kernel $P(x(t)|x(0)) = \mathcal{N}(x(0), \sigma^2(t) - \sigma^2(0))$ and $P(x(T)) \approx \mathcal{N}(0, \sigma^2(T))$. There can be multiple design choices for the noise schedule $\sigma(t)$, resulting in various formulations of diffusion models [11].

2.2. Solving Inverse problems with Diffusion models

As described in Sec. 1, solving an inverse problem entails estimation of (or sampling from) the posterior $P(x|y)$, where y is the noisy degradation of the original data sample x. In the context of solving inverse problems using diffusion models, sampling from the posterior $P(x(0)|y)$ involves conditioning the reverse diffusion process on y which translates to solving the modified reverse-SDE in Eq. (5).

$$dx = [f(x, t) - g(t)^2 \nabla_x \log P_t(x|y)]dt + g(t)d\bar{w} \tag{5}$$

Similar to methods of the first category (Sec. 1), which directly learn the posterior $P(x|y)$ as part of their training, it is possible to train a conditional diffusion model that learns the conditional score function directly. More specifically, one could learn $S_\theta(x, y, t)$ using conditional score matching

objectives in place of the usual unconditional score function $S_\theta(x, t)$ in Sec. 2.1. In this work, we focus on methods of the second category, which only leverage an unconditional diffusion model for $P(x)$, to infer $P(x|y)$. We modify the notations to denote $x(t)$ with x_t, $\sigma(t)$ with σ_t, and $P_t(x)$ with $P(x_t)$ for the sake of convenience and to be consistent with previous works [5, 21].

$$\nabla_{x_t} \log P(x_t|y) = \nabla_{x_t} \log P(x_t) + \nabla_{x_t} \log P(y|x_t) \quad (6)$$

$$P(y|x_t) = \int_{x_0} P(y|x_0)P(x_0|x_t)\mathrm{d}x_0 \quad (7)$$

Solving Eq. (5) involves estimating the conditional score function $\nabla_{x_t} \log P(x_t|y)$. The pre-trained unconditional diffusion model can be used to estimate $\nabla_{x_t} \log P(x_t)$, however, the term $\nabla_{x_t} \log P(y|x_t)$ becomes intractable (ref Eqs. (6) and (7)). At its core, the intractability of $\nabla_{x_t} \log P(y|x_t)$ arises from the fact that $P(x_0|x_t)$ is intractable [21], and hence, the conditional score is hard to estimate while only leveraging the unconditional score.

2.3. Related works

PGDM [21] approximates $P(x_0|x_t)$ with a Gaussian distribution having mean $\hat{x_t}$ and variance ${r_t}^2$, where $\hat{x_t} = \mathbb{E}(x_0|x_t) = x_t + \sigma_t^2 \nabla_{x_t} \log P(x_t)$ (using Tweedie's formula). The standard deviation r_t is a hyperparameter, chosen proportionally to σ_t. DPS [5] approximates $P(y|x_t)$ with the point estimate $P(y|x_0 = \hat{x_t})$ and has an almost similar formulation as PGDM up to a constant factor, though the motivation is slightly different. Boys et al. [3] also use Gaussian approximation but further replace r_t with the covariance matrix $\mathrm{Cov}[x_0|x_t] = \sigma_t^2 \frac{\partial \hat{x_t}}{\partial x_t}$. Computing this matrix is expensive in practice, so they resort to diagonal and row-sum approximations of the matrix instead. Peng et al. [16] proposes to find an optimal covariance matrix using learned covariances from the diffusion model. All these methods, however, still assume simplified approximations for $P(x_0|x_t)$, which limits their performance in practice, given the complicated and multimodal nature of the true data distribution. Other works [6, 7, 12, 30] try to circumvent this term by projecting the intermediate x_t onto the measurement subspace using heuristic approximations.

DiffPIR [33] poses the problem as MAP optimization with data and prior terms and utilizes the HQS [8] algorithm to solve a relaxed problem where the data and the prior terms can be optimized alternatively in a decoupled manner. Specifically, in each reverse diffusion step, this amounts to solving a relaxed MAP objective. DDS [6] uses a similar framework but employs subspace projection methods to solve the intermediate MAP objectives at each diffusion step. ZSIR [4] and DMPlug [29] also pose the problem as MAP optimization, where instead of optimizing for the original data x_0 directly, they reparameterize x_0 via the initial diffusion noise x_T and solve for the optimal noise x_T instead. From a theoretical perspective, it is unclear how this reparameterization should help. Also, both works ignore the prior term while focusing only on the data term. If the data distribution has full support, i.e. $P(x_0) \neq 0$ $\forall x_0 \in \mathbb{R}^n$, ignoring the prior term, especially in the case of a noisy measurement, is theoretically unjustified. Our proposed method is similar to ZSIR and DMPlug from a practical perspective, however, our MAP formulation is based on sound theoretical motivation that justifies the reparameterization based on PF ODE, providing new insights into modeling the conditional generation process as MAP optimization.

3. Our Methodology

3.1. Background: Consistency Models

Consider the PF ODE described in Eq. (4). The solution trajectories of this ODE are smooth, and map the samples on the data manifold to pure noise. In [23], a consistency model is defined as the function that maps any point on the PF ODE trajectory to its corresponding origin (initial point on the data manifold). There exist efficient methodologies [22] to train these consistency models in practice. Please refer to [23] for a detailed description.

3.2. Proposed MAP estimation framework

$$x_0^* = \arg\max_{x_0} \log P(x_0|y) \quad (8)$$

Eq. (8) refers to the usual MAP formulation for solving an inverse problem. Finding an optimal x_0^* using gradient ascent involves the update step in Eq. (9), with k denoting the k^{th} iterate and λ denoting the step size.

$$x_0^{k+1} = x_0^k + \lambda * \nabla_{x_0} \log P(x_0|y) \quad (9)$$

The update step requires computing the gradient term $\nabla_{x_0} \log P(x_0|y) = \nabla_{x_0} \log P(y|x_0) + \nabla_{x_0} \log P(x_0)$. The former term is tractable since $P(y|x_0)$ is Gaussian, and the latter is the score function evaluated at x_0 and can be replaced with $S_\theta(x_0, 0)$. In practice, $S_\theta(x_0, 0)$ is only accurate when x_0 lies closer to the data manifold and is typically inaccurate for x_0 in low-likelihood regions outside the data manifold [24]. This makes the score estimate inaccurate in the beginning (when x_0 is initialized randomly) and during the gradient ascent updates, since the intermediate x_0^k are not constrained to lie on the data manifold. This issue can be avoided when inverse problems are typically solved through the reverse diffusion process (Sec. 2.2) which drifts the noisy sample towards the data manifold using $S_\theta(x_t, t)$ while simultaneously ensuring measurement consistency. But there the challenge is to estimate $\nabla_{x_t} \log P(y|x_t)$, which is again intractable.

Here, we present our proposed MAP formulation. Let $z \sim P(x_T) = \mathcal{N}(0, \sigma_T^2 \mathbb{I})$, denote a purely noisy sample, and $\mathcal{M}$ denote the data Manifold. The PF ODE trajectory maps z to a sample $x_0 \in \mathcal{M}$, given by $x_0 = f_\theta(z, T)$, where f_θ is the consistency model. It is also evident that, $\forall\, x_0 \in \mathcal{M}, \exists\, z \sim P(x_T)$ such that $x_0 = f_\theta(z, T)$. Hence, the usual MAP formulation in Eq. (8) is equivalent to the proposed MAP formulation in Eqs. (10) and (11).

$$z^* = \arg\max_z \log P(x_0 = f_\theta(z, T)|y) \tag{10}$$

$$x_0^* = f_\theta(z^*, T) \tag{11}$$

With our proposed formulation, we update z with gradient-ascent steps as in Eq. (12) for finding z^*. The update step now requires computing the gradient term $\nabla_z \log P(f_\theta(z, T)|y)$, which can be reformulated as a vector-Jacobian product (vjp) as shown in Eq. (13). The vector in this vjp is the gradient term $\nabla_{x_0} \log P(x_0|y)$ evaluated at $x_0 = f_\theta(z, T)$ which lies on the data manifold (by definition of consistency model) and can be accurately evaluated, unlike the previous case.

$$z^{k+1} = z^k + \lambda * \nabla_z \log P(f_\theta(z, T)|y) \tag{12}$$

$$\nabla_z \log P(f_\theta(z, T)|y) = \left(\frac{\partial f_\theta(z,T)}{\partial z}\right)^\intercal \nabla_{x_0} \log P(x_0|y)\Big|_{x_0 = f_\theta(z,T)} \tag{13}$$

Here we provide a high-level overview of a practical implementation using our MAP formulation. In practice, even a consistency model f_θ can benefit from multi-step sampling [23]. Therefore we propose a multi-step gradient ascent scheme called ***MAP-Gradient-Ascent (MAP-GA)*** as described in Algorithm 1. Note that τ refers to a time-step schedule with $T = \tau_n > \tau_{n-1} > .. > \tau_1 > \tau_0 = 0$ and σ refers to the monotonically increasing noise schedule for $t \in [0, T]$, with $\sigma_0 = 0$, and $\sigma_T = \infty$ (high value in practice), y is the measurement, f_θ and S_θ are the consistency model and the score model respectively. num_iter denotes the number of gradient ascent iterations per time step, and λ denotes the learning rate. Algorithm 1 can be applied to solve any inverse problem effectively, given that we know the optimal hyper-parameters, such as the time-step schedule τ, the learning rate λ, learning rate schedule, num_steps, etc. However, finding those in practice can be quite challenging.

3.3. Practical Implementation

In practice, to avoid numerical issues and to ensure that the gradient term $\nabla_{x_0} \log P(x_0)$ exists, instead of x_0^*, we solve for $x_\epsilon^* = \arg\max_{x_\epsilon} \log P(x_\epsilon|y)$, for a small ϵ such that $\sigma_\epsilon \approx 0$. We solve the MAP formulation in Eqs. (14) and (15). Since, $P(x_\epsilon|x_0) = \mathcal{N}(x_0, \sigma_\epsilon^2 \mathbb{I})$, for very small values of σ_ϵ, the distinction between x_0 and x_ϵ remain insignificant for all practical purposes. The consistency models in [23] are also learned to map the points on PF ODE trajectory to the corresponding x_ϵ instead of x_0.

Algorithm 1: MAP-GA (MAP-Gradient-Ascent)

input : $\tau = (\tau_n, ..\tau_1, \tau_0)$, f_θ, S_θ, y, *num_iter*, λ, σ
$z \sim \mathcal{N}(0, \sigma_{\tau_n}^2 \mathbb{I})$
for i *in* $(n, n-1, .., 1)$ **do**
  $t = \tau_i$
  for j *in* $(1, 2, .., num\_iter)$ **do**
    $z = z + \lambda * \nabla_z \log P(f_\theta(z, t)|y)$
  end
  $\hat{x}_0 = f_\theta(z, t)$
  $z = \mathcal{N}(\hat{x}_0, \sigma_{\tau_{i-1}}^2 \mathbb{I})$
end
output: z

$$z^* = \arg\max_z \log P(x_\epsilon = f_\theta(z, T)|y) \tag{14}$$

$$x_0^* \approx x_\epsilon^* = f_\theta(z^*, T) \tag{15}$$

Here, we describe in detail the computation of the gradient term $\nabla_z \log P(f_\theta(z, t)|y)$ in practice. From Eqs. (16) and (17), this requires the estimation of the gradient of log-likelihood i.e. $\nabla_{x_\epsilon} \log P(y|x_\epsilon)$ and the gradient of log-prior i.e. $\nabla_{x_\epsilon} \log P(x_\epsilon)$. The terms $P(y|x_\epsilon)$ and $P(x_\epsilon)$ are also referred to as the **likelihood** and the **prior** respectively.

$$\nabla_z \log P(f_\theta(z, t)|y) = \left(\frac{\partial f_\theta(z,t)}{\partial z}\right)^\intercal \nabla_{x_\epsilon} \log P(x_\epsilon|y)\Big|_{x_\epsilon = f_\theta(z,t)} \tag{16}$$

$$\nabla_{x_\epsilon} \log P(x_\epsilon|y)\Big|_{x_\epsilon = f_\theta(z,t)} = \Big\{\nabla_{x_\epsilon} \log P(y|x_\epsilon) + \nabla_{x_\epsilon} \log P(x_\epsilon)\Big\}\Big|_{x_\epsilon = f_\theta(z,t)} \tag{17}$$

Computing the gradient of log-likelihood
$P(y|x_0) = \mathcal{N}(\mathcal{A}(x_0), \sigma_y^2 \mathbb{I})$, and $P(x_\epsilon|x_0) = \mathcal{N}(x_0, \sigma_\epsilon^2 \mathbb{I})$, given by Eq. (1) and the diffusion perturbation kernel respectively. Since $\sigma_\epsilon \approx 0$, $P(x_0) \approx P(x_\epsilon)$ and $P(x_0|x_\epsilon) \approx P(x_\epsilon|x_0)$. We can approximate $P(x_0|x_\epsilon) = \mathcal{N}(x_\epsilon, \sigma_\epsilon^2 \mathbb{I})$, and for a linear forward operator i.e. $\mathcal{A} = H \in \mathbb{R}^{m \times n}$, we can derive using Eq. (7), $P(y|x_\epsilon) = \mathcal{N}(Hx_\epsilon, \sigma_y^2 \mathbb{I} + \sigma_\epsilon^2 HH^\intercal)$. For non-linear $\mathcal{A}$, similar approximations can be made by linearizing it around x_ϵ. Given the tractable form of the likelihood term above, the gradient of the log-likelihood is apparent.

Computing the gradient of log-prior
The gradient of the log prior i.e. $\nabla_{x_\epsilon} \log P(x_\epsilon)$ is essentially the score function evaluated at x_ϵ. Given a score function $S_\theta(x, t)$, learned by the unconditional diffusion model, $\nabla_{x_\epsilon} \log P(x_\epsilon) = S_\theta(x_\epsilon, \epsilon)$. Learning the score function is equivalent to learning a denoiser, and vice-versa [11]. Hereon, we denote the unconditional diffusion model as learning the denoiser $D_\theta(x, t)$, from which the score function can be computed as $\nabla_{x_t} \log P(x_t) = \frac{D_\theta(x_t, t) - x_t}{\sigma_t^2}$.

4. Image restoration with MAP-GA

Several image restoration tasks such as inpainting, deblurring, and super-resolution can be modeled as linear inverse problems. For example, in image inpainting [15, 31], given a masked image (and the corresponding binary mask), the goal is to recover (reconstruct) the missing pixels of the masked image. Let $x \in \mathbb{R}^n$ denote the original image, $y \in \mathbb{R}^m$ denote a masked image with only visible pixels ($m \leq n$) and $H \in \mathbb{R}^{m \times n}$ denotes a corresponding linear forward operator for a given mask. The inpainting problem is characterized by $y = Hx + \eta$, where $\eta \sim \mathcal{N}(0, \sigma_y^2 \mathbb{I})$. Note that for inpainting, a given mask defines H with a defined structure, where the rows of H are one-hot and are orthogonal i.e. $HH^\intercal = \mathbb{I}_{m \times m}$. Other image restoration problems such as deblurring and super-resolution can be modeled accordingly with their corresponding forward operators.

We expand Algorithm 1 for image restoration and include all the specific details in Algorithm 2. The core term in the algorithm that needs to be evaluated is $\nabla_z \log P(f_\theta(z,t)|y)$, which involves the estimation of gradients of the log-likelihood and the log-prior terms (Eqs. (16) and (17)). In the algorithm, we make a choice (indicated by the use_prior keyword) of retaining or dropping the gradient of the log-prior. Dropping the prior term implies a choice of uniform prior, and the algorithm now optimizes for the maximum likelihood estimate instead of the MAP estimate, by considering any sample (consistent with our measurement) on the data manifold to be equally good. This is also the setting considered in the concurrent works [4, 29].

Note that the algorithm makes use of both the consistency model (C_θ) and the denoiser (D_θ), which makes our method more demanding compared to other methods that only use the denoiser. We make an argument as follows. We use the pre-trained denoiser and the consistency model from [11] and [23] respectively, with noise schedule $\sigma(t) = t$ and $t \in [\epsilon, T]$. Consider the corresponding PF ODE (from Eq. (4)) for the above setting, as follows.

$$\mathrm{d}x = -\frac{D_\theta(x,t) - x}{t}\mathrm{d}t$$

This PF ODE determines the trajectory and solving the trajectory origin x_0 involves solving the ODE above. Note that this x_0 is what the consistency model C_θ is trained to predict. As a rough approximation, solving the ODE with a backward Euler discretization step from t to 0, which essentially assumes the trajectory curves are linear, i.e. the Jacobian $\frac{\mathrm{d}x}{\mathrm{d}t}$ is constant for the interval $[0, t]$, and this gives $C_\theta(x_t, t) = x_0 \approx D_\theta(x_t, t)$, with the approximation getting more accurate as the trajectory curve gets more linear. In an empirical setting, [11] also observes trajectories become more linear when $\sigma(t) \to 0$. While more analysis on this is still due, it motivates us to look at the denoiser as a proxy of the consistency model, which gradually becomes more and more accurate as $\sigma(t) \to 0$. Hence, we also consider settings that replace the consistency model with the denoiser in our algorithm.

Algorithm 2: MAP-GA for Image restoration

input : time schedule: $\tau = [\, \tau_n, \tau_{n-1}, ..\tau_1, \tau_0 \,]$,
noise schedule: $\sigma(.)$, denoiser: D_θ,
consistency model: C_θ,
measurement: y, learning rate: λ,
num gradient ascent iter: num_iter,
boolean: use_prior (default True),
forward operator matrix: H,
measurement noise: σ_y
(Note: $\tau_0 = \epsilon$, $\tau_n = T$),

$z \sim \mathcal{N}(0, \sigma_{\tau_n}^2 \mathbb{I})$
for i *in* $(\, n, n-1, \, .. \, 1\,)$ **do**
  $t = \tau_i$
  for j *in* $(\, 1, 2, \, .. \, num\_iter\,)$ **do**
    $\hat{x}_\epsilon = C_\theta(z, t)$
    $grad_{likelihood} = \frac{H^\intercal(\frac{\sigma_y^2}{\sigma_\epsilon^2}\mathbb{I} + HH^\intercal)^{-1}(y - H\hat{x}_\epsilon)}{\sigma_\epsilon^2}$
    if *use_prior* **then**
      $grad_{prior} = \frac{D_\theta(\hat{x}_\epsilon, \epsilon) - \hat{x}_\epsilon}{\sigma_\epsilon^2}$
    end
    else
      $grad_{prior} = 0$
    end
    $grad_{posterior} = grad_{likelihood} + grad_{prior}$
    $grad = \left(\frac{\partial C_\theta(z,t)}{\partial z}\right)^\intercal grad_{posterior}$
    $z = z + \lambda * grad$
  end
  $\hat{x}_\epsilon = C_\theta(z, t)$
  $z = \mathcal{N}(\hat{x}_\epsilon, \sigma_{\tau_{i-1}}^2 - \sigma_{\tau_0}^2 \mathbb{I})$
end
output: z

When using Algorithm 2 for noisy image restoration ($\sigma_y > 0$) in practice, we observe that it requires careful tuning of the learning rate and other hyperparameters. To avoid such sensitive hyperparameters, we present Algorithm 3 for noisy image restoration, based on our empirical observations from Tab. 3. The motivation for Algorithm 3, is to find an approximate solution using MAP-GA at diffusion time $t = \tau$ where $\sigma_\tau = \sigma_y$, and use it as an initialization for PGDM [21] for $\sigma_t < \sigma_y$. Specifically, we use MAP-GA until $\sigma_t = \sigma_y$, to find x_{σ_y} and later use this as an initialization to PGDM for $\sigma_t < \sigma_y$. We do not use the prior term for the MAP-GA part in Algorithm 3, as we find it more effective.

Algorithm 3: MAP-GA-PGDM Image restoration

input : noise schedule: $\sigma(.)$, denoiser: D_θ,
consistency model: C_θ,
measurement: y, learning rate: λ,
num gradient ascent iter: num_iter,
forward operator matrix: H,
measurement noise: σ_y
$\tau^{map-ga} = [\tau_n^{map}, \tau_{n-1}^{map}, ..\tau_1^{map}, \tau_0^{map}]$,
$\tau^{pgdm} = [\tau_m^{pgdm}, \tau_{m-1}^{pgdm}, ..\tau_1^{pgdm}, \tau_0^{pgdm}]$,
(Note: $\sigma_{\tau_0^{map}} = \sigma_y$ and $\tau_n^{map} = T$)
(Note: $\sigma_{\tau_m^{pgdm}} = \sigma_y$ and $\tau_0^{pgdm} = \epsilon$),

=======MAP-GA=======

$z \sim \mathcal{N}(0, \sigma^2_{\tau_n^{map}}\mathbb{I})$

for i *in* $(\, n, n-1, \, .. \, 1\,)$ **do**

  $t = \tau_i^{map}$

  for j *in* $(\, 1, 2, \, .. \, num\_iter\,)$ **do**

    $\hat{x}_\epsilon = C_\theta(z, t)$

    $grad_{likelihood} = \frac{H^\intercal(\frac{\sigma_y^2}{\sigma_\epsilon^2}\mathbb{I} + HH^\intercal)^{-1}(y - H\hat{x}_\epsilon)}{\sigma_\epsilon^2}$

    $grad_{posterior} = grad_{likelihood}$

    $grad = \left(\frac{\partial C_\theta(z,t)}{\partial z}\right)^\intercal grad_{posterior}$

    $z = z + \lambda * grad$

  end

  $\hat{x}_\epsilon = C_\theta(z, t)$

  $z = \mathcal{N}(\hat{x}_\epsilon, \sigma^2_{\tau_{i-1}^{map}} - \sigma^2_{\tau_0^{map}}\mathbb{I})$

end

========PGDM========

$x_{\tau_m^{pgdm}} = z$

for i *in* $(\, m, m-1, \, .. \, 1\,)$ **do**

  $t = \tau_i^{pgdm}$

  $\hat{x}_t = D_\theta(z, t)$

  $\mu_t = \hat{x}_t + \sigma_t^2 * \nabla_{x_t} \log P(y|x_t)$ *pgdm update*

  $x_{\tau_{i-1}^{pgdm}} = \mathcal{N}(\mu_t, \sigma^2_{\tau_{i-1}^{pgdm}} - \sigma^2_{\tau_0^{pgdm}}\mathbb{I})$

end

output: $x_{\tau_0^{pgdm}}$

5. Experiments

We consider the tasks of image inpainting, deblurring, and $4\times$ super-resolution. Inspired by [13], for image inpainting, we evaluate the performance across six different mask settings (Fig. 1) denoting varying levels of degradation. The mask settings *box50* and *box25* indicate a square crop at the center of the image, with the crop width equal to *50%* and *25%* of the image width respectively. In *half*, we mask out the right half of the image, *expand* is the complement of *box25*. *sr2x* denotes a $2\times$ super-resolution mask, and *altlines* masks out alternate rows of pixels.

We evaluate the performance of MAP-GA (Algorithm 2) on ImageNet [18] 1K validation set with 64×64 resolution (in Tab. 1) and on 100 random images of LSUNCat [32] with 256×256 resolution (in Tab. 2). We use the pre-trained denoisers and the consistency models from [11, 23] with their default settings. In the experiments, the setting MAP-GA denotes the default Algorithm 2, MAP-GA(NP) denotes MAP-GA with no prior, MAP-GA(D) denotes MAP-GA with denoiser replacing the consistency model, and MAP-GA(D,NP) denotes MAP-GA with no prior and the denoiser replacing the consistency model. We compare against DDRM [12] and PGDM [21] (both only use the denoiser) and against the zero-shot image editing (CT-ZSIE) algorithm proposed in [23] which only use the consistency model. The results from Tabs. 1 and 2 show MAP-GA and variants outperform DDRM, PGDM, and CT-ZSIE with a significant margin on several tasks.

MAP-GA uses gradient ascent (first-order gradient-based method) to optimize the underlying MAP objective, which is typically highly non-convex and is not guaranteed to find the global optima. However, it could converge with an initialization closer to the global optima. To corroborate this, we design the following toy experiment. We consider the noiseless inpainting task from earlier, but, in Algorithm 2, we set $\tau_n = \hat{\tau} \ll T$, instead of the default setting $\tau_n = T$ and we also initialize $z \sim \mathcal{N}(x_0, \sigma_{\hat{\tau}}^2\mathbb{I})$, where x_0 is the corresponding ground truth image for the measurement y. From Tab. 3, we observe significant improvements in the performance. (We use $\hat{\tau} = 0.5$, note that $\sigma_{0.5} = 0.5$, as we use the noise schedule $\sigma_t = t$). This shows that MAP-GA is only limited by the choice of optimizer and reinforces the need for better optimization algorithms. While it is important to consider better design choices (for the schedules and hyperparameters), adaptive-gradient-based optimizers (such as momentum, Adam), or higher-order methods, we leave this for future work as it requires a thorough analysis.

In Tab. 4, we report the results on noisy inpainting using Algorithm 3 on the ImageNet64 1K validation set. In Algorithm 3, and all its variants, we fix $m = 20$ time steps for PGDM. Even with high levels of measurement noise, MAP-GA-PGDM shows promising improvements over PGDM. In all our experiments, for MAP-GA and variants, we fix a budget of 1000 steps and run ablations for (num_steps, num_iter) from the set $\{$(20,50),(50,20),(100,10),(200,5),(250,4),(500,2),(1000,1)$\}$. For DDRM, PGDM, and CT-ZSIE, we run ablations for num_steps from the set $\{$20,50,100,200,250,500,1000$\}$. The learning rate in all our experiments was set to $\sigma_y^2 + \sigma_\epsilon^2$.

Method	box50		half		expand		box25		sr2x		altlines		deblur		supres4x	
	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓
MAP-GA(D,NP)	**32.312**	**0.096**	**37.986**	**0.157**	93.068	0.419	**11.019**	**0.019**	**30.072**	**0.034**	**18.282**	**0.015**	**19.704**	**0.007**	66.968	0.148
MAP-GA(D)	35.761	0.100	43.315	0.164	106.32	0.430	11.169	0.019	33.188	0.036	19.103	0.016	19.779	0.008	78.791	0.179
MAP-GA(NP)	34.944	0.113	39.243	0.162	**69.004**	**0.388**	12.818	0.027	30.303	0.035	20.321	0.018	22.090	0.008	**46.349**	**0.112**
MAP-GA	36.733	0.113	41.151	0.164	87.952	0.400	12.836	0.027	33.752	0.036	20.895	0.018	21.314	0.010	59.624	0.120
PGDM [21]	49.370	0.151	54.261	0.245	127.95	0.479	14.255	0.021	38.433	0.046	20.446	0.019	19.857	0.007	89.614	0.238
DDRM [12]	51.477	0.165	56.643	0.264	136.06	0.492	15.000	0.023	35.033	0.041	19.331	0.017	23.195	0.009	78.712	0.235
CT-ZSIE [23]	38.017	0.129	44.152	0.191	70.634	0.424	13.040	0.025	42.500	0.060	26.737	0.029	29.223	0.013	56.698	0.116

Table 1. Noiseless image restoration on ImageNet64 1K validation set using MAP-GA and variants. The setting MAP-GA denotes the default Algorithm 2, MAP-GA(NP) denotes MAP-GA with no prior, MAP-GA(D) denotes MAP-GA with denoiser replacing the consistency model, and MAP-GA(D,NP) denotes MAP-GA with no prior and the denoiser replacing the consistency model.

Method	box50		half		expand		box25		sr2x		altlines		deblur		supres4x	
	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓
MAP-GA(D,NP)	**70.880**	**0.128**	**68.022**	**0.269**	174.96	0.695	18.221	0.028	**22.819**	0.059	13.632	0.036	81.461	0.214	59.895	0.205
MAP-GA(D)	74.995	0.137	75.748	0.280	197.91	0.696	**16.795**	**0.025**	24.420	0.063	13.669	0.033	86.585	0.220	63.308	0.214
MAP-GA(NP)	98.749	0.165	86.975	0.297	**155.55**	**0.645**	30.788	0.046	24.434	**0.054**	16.126	0.034	**80.337**	**0.211**	**49.622**	**0.146**
MAP-GA	108.87	0.171	85.936	0.291	175.27	0.652	34.711	0.053	25.666	0.056	15.139	0.036	84.971	0.214	68.156	0.169
PGDM [21]	120.82	0.194	94.920	0.360	227.97	0.765	28.357	0.041	27.927	0.070	14.211	0.037	94.629	0.227	77.533	0.248
DDRM [12]	131.86	0.198	101.86	0.379	224.93	0.778	29.571	0.041	23.686	0.065	**12.040**	**0.026**	105.28	0.266	75.923	0.251
CT-ZSIE [23]	118.27	0.209	121.11	0.375	200.03	0.704	34.246	0.046	47.353	0.145	24.663	0.070	97.343	0.264	50.608	0.157

Table 2. Noiseless image restoration on 100 LSUNCat256 images using MAP-GA and variants. The setting MAP-GA denotes the default Algorithm 2, MAP-GA(NP) denotes MAP-GA with no prior, MAP-GA(D) denotes MAP-GA with denoiser replacing the consistency model, and MAP-GA(D,NP) denotes MAP-GA with no prior and the denoiser replacing the consistency model.

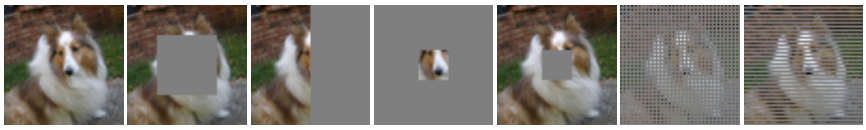

Figure 1. Left to right: original image, and mask settings: box50, half, expand, box25, sr2x, altlines

6. Discussion

6.1. Runtime comparison

Tab. 5 compares the wall-clock time of MAP-GA and variants against DDRM, PGDM, and CT-ZSIE for image inpainting on ImageNet64. To ensure a fair comparison, we keep the batch size fixed at 50 for all the methods and compare their runtime per iteration. For DDRM, CT-ZSIE, and PGDM, it is the total runtime divided by num_steps i.e. the reverse diffusion time steps, while for MAP-GA and variants, it is the effective runtime per num_steps per num_iter (i.e. it is the runtime when $num\_steps = num\_iter = 1$). MAP-GA and variants are $1.5\times$ to $2\times$ slower per iteration than PGDM and $3\times$ to $4\times$ slower than DDRM and CT-ZSIE.

6.2. Concurrent works

The algorithms presented in this paper are similar to ZSIR [4], and DMPlug [29] from a practical perspective. However, our MAP formulation is novel, has a strong theoretical motivation, and connects the PF ODE, the consistency model with the MAP optimization for solving inverse problems. Unlike ZSIR and DMPlug, we consider the prior term and show that the gradient of the log-prior is tractable, making the gradient of the log-posterior tractable. We show that MAP-GA is only limited by the optimizer's choice in practice. ZSIR and DMPlug replace the PF ODE trajectory origin with a multi-step denoiser approximation and require backpropagation through the chain of cascaded functions to optimize the loss. In contrast, MAP-GA variants require a single vector-Jacobian product per iteration.

7. Conclusion

In this paper, we proposed a novel MAP formulation for solving inverse problems using pre-trained unconditional diffusion models. Note that conditional generation is a core requirement in solving inverse problems. We connect the Probability Flow ODE and the consistency model with the optimization process for the MAP objective in tasks that involve conditional generation. We showed that the gradi-

Method	box50		half		expand		box25		sr2x		altlines	
	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓
MAP-GA(D,NP)	26.661	0.043	30.621	0.064	73.510	0.155	**9.165**	0.010	29.878	0.027	**16.971**	**0.012**
MAP-GA(D)	34.689	0.052	41.098	0.083	90.768	0.175	10.891	0.011	34.334	0.031	18.718	0.013
MAP-GA(NP)	**25.205**	**0.043**	**27.208**	**0.047**	**42.671**	**0.075**	10.936	0.017	**27.496**	**0.023**	18.732	0.013
MAP-GA	29.508	0.045	33.653	0.050	73.318	0.106	11.257	0.016	34.364	0.026	20.041	0.013
PGDM [21]	31.952	0.056	36.639	0.082	71.352	0.138	10.886	**0.009**	33.199	0.041	19.551	0.018

Table 3. Noiseless inpainting on ImageNet64 1K validation set. Using the ground truth image (x_0) for the measurement y, we create a sample at $t = 0.5$ via ($x_{0.5} = x_0 + 0.5 * \eta$, where, $\eta \sim \mathcal{N}(0, \mathbb{I})$) and initialize Algorithm 2 with $z = x_{0.5}$, and $\tau_n = 0.5$

Method	σ_y	box50		half		expand		box25		sr2x		altlines	
		FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓	FID↓	LPIPS↓
MAP-GA-PGDM(D)	0.05	**56.173**	**0.125**	62.026	0.193	108.16	0.438	**38.725**	**0.039**	57.41	0.080	46.046	0.046
MAP-GA-PGDM	0.05	58.283	0.147	**61.588**	**0.184**	**91.508**	**0.406**	44.667	0.085	**57.308**	**0.073**	**45.265**	**0.042**
PGDM [21]	0.05	77.824	0.175	80.248	0.257	135.99	0.495	53.136	0.049	86.289	0.126	66.203	0.066
MAP-GA-PGDM(D)	0.1	**72.543**	**0.166**	79.535	0.230	114.51	0.464	**57.556**	**0.076**	76.733	0.145	65.154	0.096
MAP-GA-PGDM	0.1	74.130	0.191	**78.322**	**0.225**	**103.85**	**0.440**	65.720	0.161	**76.248**	**0.134**	**63.134**	**0.089**
PGDM [21]	0.1	96.485	0.216	99.170	0.286	145.40	0.519	78.620	0.100	109.47	0.231	90.925	0.138

Table 4. Noisy inpainting on ImageNet64 1K validation set. σ_y denotes the measurement noise. The setting MAP-GA-PGDM denotes the default Algorithm 3, MAP-GA-PGDM(D) denote MAP-GA-PGDM with denoiser replacing the consistency model.

Method	Runtime per iteration
DDRM [12]	150 ms
CT-ZSIE [23]	150 ms
PGDM [21]	304 ms
MAP-GA(D,NP)	456 ms
MAP-GA(D)	602 ms
MAP-GA(NP)	455 ms
MAP-GA	603 ms

Table 5. Runtime comparison on NVIDIA A100 40GB GPU

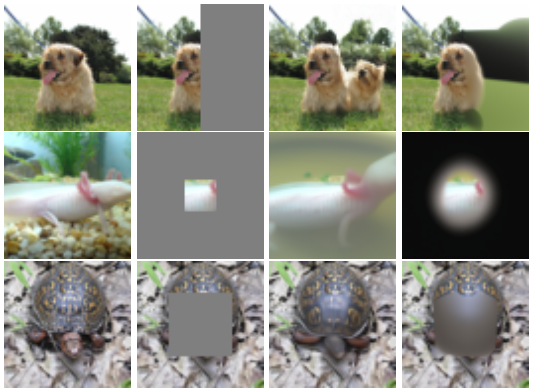

Figure 2. Noiseless inpainting task. Left to right: original image, masked image, restored images using MAP-GA, PGDM.

ent of the MAP objective is tractable, allowing the use of gradient-based optimization methods. To use our framework in practice, we proposed an algorithm with a multi-step gradient ascent strategy for MAP optimization. We validated our algorithms with extensive experiments on image deblurring, super-resolution, and inpainting.

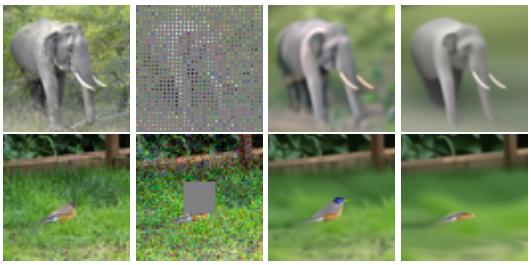

Figure 3. Noisy inpainting task ($\sigma_y = 0.1$). Left to right: original, masked image, restored images using MAP-GA-PGDM, PGDM.

Acknowledgements

This work was funded by the Marie Skłodowska-Curie Actions project MODELAIR, through grant agreement no. 101072559. The computations and the data handling were enabled by resources provided by the National Academic Infrastructure for Supercomputing in Sweden (NAISS), partially funded by the Swedish Research Council through grant agreement no. 2022-06725. Bharath thanks Sebastian Gerard and Heng Fang for their feedback on improving the paper presentation.

References

[1] Simon Arridge, Peter Maass, Ozan Öktem, and Carola-Bibiane Schönlieb. Solving inverse problems using data-driven models. *Acta Numerica*, 28:1–174, 2019. 1

[2] Ashish Bora, Ajil Jalal, Eric Price, and Alexandros G Dimakis. Compressed sensing using generative models. In *International conference on machine learning*, pages 537–546. PMLR, 2017. 1

[3] Benjamin Boys, Mark Girolami, Jakiw Pidstrigach, Sebastian Reich, Alan Mosca, and O Deniz Akyildiz. Tweedie moment projected diffusions for inverse problems. *arXiv preprint arXiv:2310.06721*, 2023. 3

[4] Hamadi Chihaoui, Abdelhak Lemkhenter, and Paolo Favaro. Zero-shot image restoration via diffusion inversion, 2024. 3, 5, 7

[5] Hyungjin Chung, Jeongsol Kim, Michael Thompson Mccann, Marc Louis Klasky, and Jong Chul Ye. Diffusion posterior sampling for general noisy inverse problems. In *International Conference on Learning Representations*, 2023. 1, 3

[6] Hyungjin Chung, Suhyeon Lee, and Jong Chul Ye. Decomposed diffusion sampler for accelerating large-scale inverse problems. *arXiv preprint arXiv:2303.05754*, 2023. 3

[7] Hyungjin Chung, Byeongsu Sim, Dohoon Ryu, and Jong Chul Ye. Improving diffusion models for inverse problems using manifold constraints. *Advances in Neural Information Processing Systems*, 35:25683–25696, 2022. 3

[8] Donald Geman and Chengda Yang. Nonlinear image recovery with half-quadratic regularization. *IEEE transactions on image processing : a publication of the IEEE Signal Processing Society*, 4 7:932–46, 1995. 3

[9] Ian Goodfellow, Jean Pouget-Abadie, Mehdi Mirza, Bing Xu, David Warde-Farley, Sherjil Ozair, Aaron Courville, and Yoshua Bengio. Generative adversarial nets. *Advances in neural information processing systems*, 27, 2014. 1

[10] Jonathan Ho, Ajay Jain, and Pieter Abbeel. Denoising diffusion probabilistic models. *Advances in neural information processing systems*, 33:6840–6851, 2020. 2

[11] Tero Karras, Miika Aittala, Timo Aila, and Samuli Laine. Elucidating the design space of diffusion-based generative models. *Advances in neural information processing systems*, 35:26565–26577, 2022. 2, 4, 5, 6

[12] Bahjat Kawar, Michael Elad, Stefano Ermon, and Jiaming Song. Denoising diffusion restoration models. *Advances in Neural Information Processing Systems*, 35:23593–23606, 2022. 1, 3, 6, 7, 8

[13] Andreas Lugmayr, Martin Danelljan, Andres Romero, Fisher Yu, Radu Timofte, and Luc Van Gool. Repaint: Inpainting using denoising diffusion probabilistic models. In *Proceedings of the IEEE/CVF conference on computer vision and pattern recognition*, pages 11461–11471, 2022. 6

[14] Alex Nichol, Prafulla Dhariwal, Aditya Ramesh, Pranav Shyam, Pamela Mishkin, Bob McGrew, Ilya Sutskever, and Mark Chen. Glide: Towards photorealistic image generation and editing with text-guided diffusion models. *arXiv preprint arXiv:2112.10741*, 2021. 1

[15] Deepak Pathak, Philipp Krahenbuhl, Jeff Donahue, Trevor Darrell, and Alexei A Efros. Context encoders: Feature learning by inpainting. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 2536–2544, 2016. 5

[16] Xinyu Peng, Ziyang Zheng, Wenrui Dai, Nuoqian Xiao, Chenglin Li, Junni Zou, and Hongkai Xiong. Improving diffusion models for inverse problems using optimal posterior covariance. In *Forty-first International Conference on Machine Learning*, 2024. 3

[17] Danilo Rezende and Shakir Mohamed. Variational inference with normalizing flows. In *International conference on machine learning*, pages 1530–1538. PMLR, 2015. 1

[18] Olga Russakovsky, Jia Deng, Hao Su, Jonathan Krause, Sanjeev Satheesh, Sean Ma, Zhiheng Huang, Andrej Karpathy, Aditya Khosla, Michael Bernstein, et al. Imagenet large scale visual recognition challenge. *International journal of computer vision*, 115:211–252, 2015. 6

[19] Chitwan Saharia, William Chan, Huiwen Chang, Chris Lee, Jonathan Ho, Tim Salimans, David Fleet, and Mohammad Norouzi. Palette: Image-to-image diffusion models. In *ACM SIGGRAPH 2022 conference proceedings*, pages 1–10, 2022. 1

[20] Jascha Sohl-Dickstein, Eric Weiss, Niru Maheswaranathan, and Surya Ganguli. Deep unsupervised learning using nonequilibrium thermodynamics. In *International conference on machine learning*, pages 2256–2265. PMLR, 2015. 2

[21] Jiaming Song, Arash Vahdat, Morteza Mardani, and Jan Kautz. Pseudoinverse-guided diffusion models for inverse problems. In *International Conference on Learning Representations*, 2023. 1, 3, 5, 6, 7, 8

[22] Yang Song and Prafulla Dhariwal. Improved techniques for training consistency models. *arXiv preprint arXiv:2310.14189*, 2023. 3

[23] Yang Song, Prafulla Dhariwal, Mark Chen, and Ilya Sutskever. Consistency models. *arXiv preprint arXiv:2303.01469*, 2023. 2, 3, 4, 5, 6, 7, 8

[24] Yang Song and Stefano Ermon. Generative modeling by estimating gradients of the data distribution. *Advances in neural information processing systems*, 32, 2019. 3

[25] Yang Song, Sahaj Garg, Jiaxin Shi, and Stefano Ermon. Sliced score matching: A scalable approach to density and score estimation. In *Uncertainty in Artificial Intelligence*, pages 574–584. PMLR, 2020. 2

[26] Yang Song, Jascha Sohl-Dickstein, Diederik P Kingma, Abhishek Kumar, Stefano Ermon, and Ben Poole. Score-based generative modeling through stochastic differential equations. *arXiv preprint arXiv:2011.13456*, 2020. 2

[27] Pascal Vincent. A connection between score matching and denoising autoencoders. *Neural computation*, 23(7):1661–1674, 2011. 2

[28] Ricardo Vinuesa and Steven L Brunton. Enhancing computational fluid dynamics with machine learning. *Nature Computational Science*, 2(6):358–366, 2022. 1

[29] Hengkang Wang, Xu Zhang, Taihui Li, Yuxiang Wan, Tiancong Chen, and Ju Sun. Dmplug: A plug-in method for solv-

ing inverse problems with diffusion models. *arXiv preprint arXiv:2405.16749*, 2024. 1, 3, 5, 7

[30] Yinhuai Wang, Jiwen Yu, and Jian Zhang. Zero-shot image restoration using denoising diffusion null-space model. *arXiv preprint arXiv:2212.00490*, 2022. 3

[31] Raymond A Yeh, Chen Chen, Teck Yian Lim, Alexander G Schwing, Mark Hasegawa-Johnson, and Minh N Do. Semantic image inpainting with deep generative models. In *Proceedings of the IEEE conference on computer vision and pattern recognition*, pages 5485–5493, 2017. 5

[32] Fisher Yu, Yinda Zhang, Shuran Song, Ari Seff, and Jianxiong Xiao. Lsun: Construction of a large-scale image dataset using deep learning with humans in the loop. *ArXiv*, abs/1506.03365, 2015. 6

[33] Yuanzhi Zhu, Kai Zhang, Jingyun Liang, Jiezhang Cao, Bihan Wen, Radu Timofte, and Luc Van Gool. Denoising diffusion models for plug-and-play image restoration. In *Proceedings of the IEEE/CVF Conference on Computer Vision and Pattern Recognition*, pages 1219–1229, 2023. 3